

Chapter5

# 클래스 용어 정리

클래스를 간단히 구현해보고, 용어들을 정리해본다.

# Marin Class 제작

## 용어 이해하기

- 클래스 / 인스턴스 / 필드 / 메서드

```
public static void main(String[] args) {  
    // TODO Auto-generated method stub  
  
    Marin m1 = new Marin();  
  
    System.out.println(m1.hp);  
    m1.run();  
    m1.run();  
    m1.run();  
    m1.run();  
    m1.run();  
}
```

Main.java

```
public class Marin {  
  
    int hp = 100;  
  
    void run()  
    {  
        hp -= 10;  
        System.out.println("RUN " + hp);  
    }  
}
```

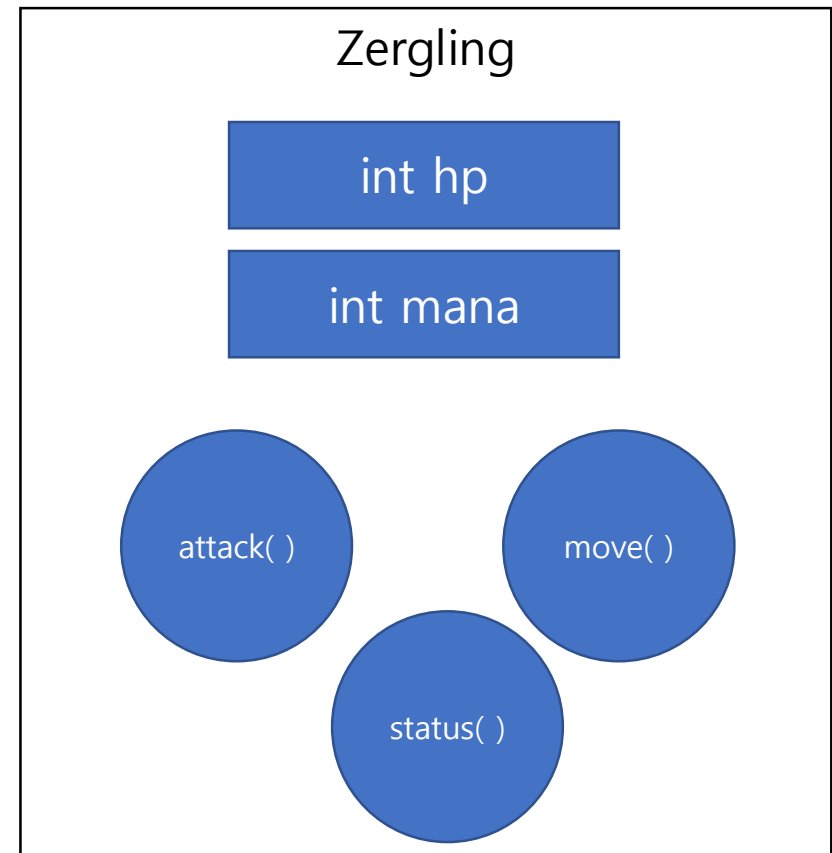
Marin.java

# [도전] Class / Instance 생성

## Zergling Class 제작

- hp, mana 기본 값 : 80, 200
- attack 메서드
  - hp가 1 증가
  - mana가 10 감소
- move 메서드
  - hp가 10 감소
  - mana 5 증가
- status 메서드
  - 현재 hp와 mana 값 화면 출력

Instance 2개 생성 후, 동작 테스트 하기



# Server Code와 Client Code

## Server Code

- Client 요청을 받으면, 처리해주는 코드
- Library == Server Code

## Client Code

- Server Code에게 일을 요청하는 코드
- Library 사용자 == Client Code

# [도전] Calculator 제작

## Server Code 구현

- Calculator Class 제작
- plus / minus / divide / multiple 메서드
  - 두 수를 받고, 계산 결과를 result 필드에 저장
- printResult 메서드
  - result 결과를 화면에 출력

## Client Code 구현

- 인스턴스 생성 후, 메서드 호출
- printResult 메서드 호출하여 결과 확인

| Calculator                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------|
| + result: int                                                                                                                         |
| + plus(int, int): void<br>+ minus(int, int): void<br>+ divide(int, int): void<br>+ multiple(int, int): void<br>+ printResult( ): void |

Chapter6

# 안전한 모듈 만들기

Client Code 관점에서 안전한 모듈을 만들어본다.

# Client들은 Readme를 읽지 않는다.

## **읽지 않는 Readme 내용**

- 해당 클래스를 사용할 때, divide에 / 0 이 되지 않도록 한다.
- result 변수를 사용하지 않는다.

Server Code Level 에서 이를 제한해야 한다.

# Calculator Class 개선 1

private로 접근하지 못하도록 막아줌

| Calculator                                                                                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------|
| - result: int                                                                                                                         |
| + plus(int, int): void<br>+ minus(int, int): void<br>+ divide(int, int): void<br>+ multiple(int, int): void<br>+ printResult( ): void |

```
public class Main {  
  
    public static void main(String[] args) {  
        Calculator c = new Calculator();  
  
        c.plus(3, 5);  
        c.printResult();  
    }  
}
```

```
public class Calculator {  
  
    private int result;  
  
    public void plus(int a, int b) { result = a + b; }  
    public void minus(int a, int b) { result = a - b; };  
    public void divide(int a, int b) { result = a / b; };  
    public void multiple(int a, int b) { result = a * b; };  
  
    public void printResult() {  
        System.out.println("결과 : " + result);  
    }  
}
```



# Calculator Class 개선 2

Client 가 어떻게 사용하더라도, 버그가 발생하지 않도록, 처리

```
public void divide(int a, int b) {  
    result = a / b;  
};
```

```
public void divide(int a, int b) {  
    if (b == 0) {  
        System.out.println("ERROR");  
        return;  
    }  
    result = a / b;  
};
```

# [도전] GameMachine Class 제작

캡슐화

**inputCoin(집어 넣을 코인 수)**

- 코인은 최대 5 개까지 넣을 수 있음
- 입력된 코인이 10 보다 초과될 수 없음

**playGame( )**

- 1 코인 씩 감소됨

집어넣은 코인이 얼마나 되는지  
확인할 수 있어야 함

- 메서드 추가 생성 필요

| GameMachine                                    |
|------------------------------------------------|
| + totalCoin: int                               |
| + inputCoin(int) : void<br>+ playGame( ): void |

**클린코드는 안전성이 높은 코드이다.**

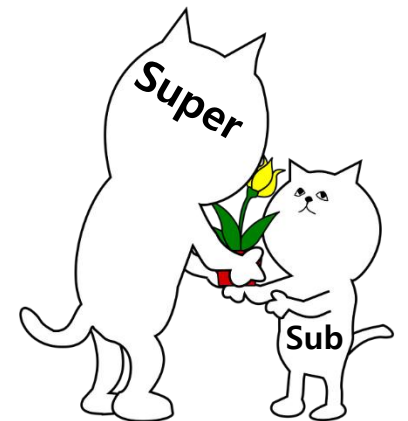
**안정성을 위해 다음과 같은 코드를 도입한다.**

1. 파라미터 유효성 검사 (입력 검증, Valid Check)
2. Precondition Check
3. 접근이 불필요한 멤버를 `private` 처리 (접근차단)

Chapter7

# 상속과 다형성

상속과 다형성의 필요성 이해



# 상속의 개념

## 상속

- 부모가 가진 요소들을  
자식들이 물려받아 사용할 수 있음

## OOP 상속

- 부모 / 자식 관계로 보기 어려움
- 코드 중복 방지를 위해 공통적인 요소를 일반화 시킴

```
class Machine {  
    String ver;  
}  
  
class Robot extends Machine {  
  
    void run()  
    {  
        System.out.println("HI");  
    }  
}  
  
public class Main {  
  
    public static void main(String[] args) {  
        Robot a = new Robot();  
  
        a.run();  
        a.run();  
        a.ver = "1.99";  
  
    }  
}
```

# [도전] 다음과 같은 코드 제작

## 3개의 클래스 제작

| SpeedRobot                                      |
|-------------------------------------------------|
| + hp: int<br>+ modelID: int                     |
| + move( )<br>+ stop( )<br>+ run( )<br>+ walk( ) |

| PowerRobot                                         |
|----------------------------------------------------|
| + hp: int<br>+ mana: int                           |
| + move( )<br>+ stop( )<br>+ attack( )<br>+ jump( ) |

| SmartRobot                                                         |
|--------------------------------------------------------------------|
| + hp: int<br>+ IQ : int                                            |
| + move( )<br>+ stop( )<br>+ fly( )<br>+ sitDown( )<br>+ standUp( ) |

# 중복 코드의 문제점

## 중복된 메서드 / 필드들이 존재

- 중복 코드가 문제가 되는 이유 :  
변경시 모두 다 한꺼번에 수정 필요 / 버그 유발

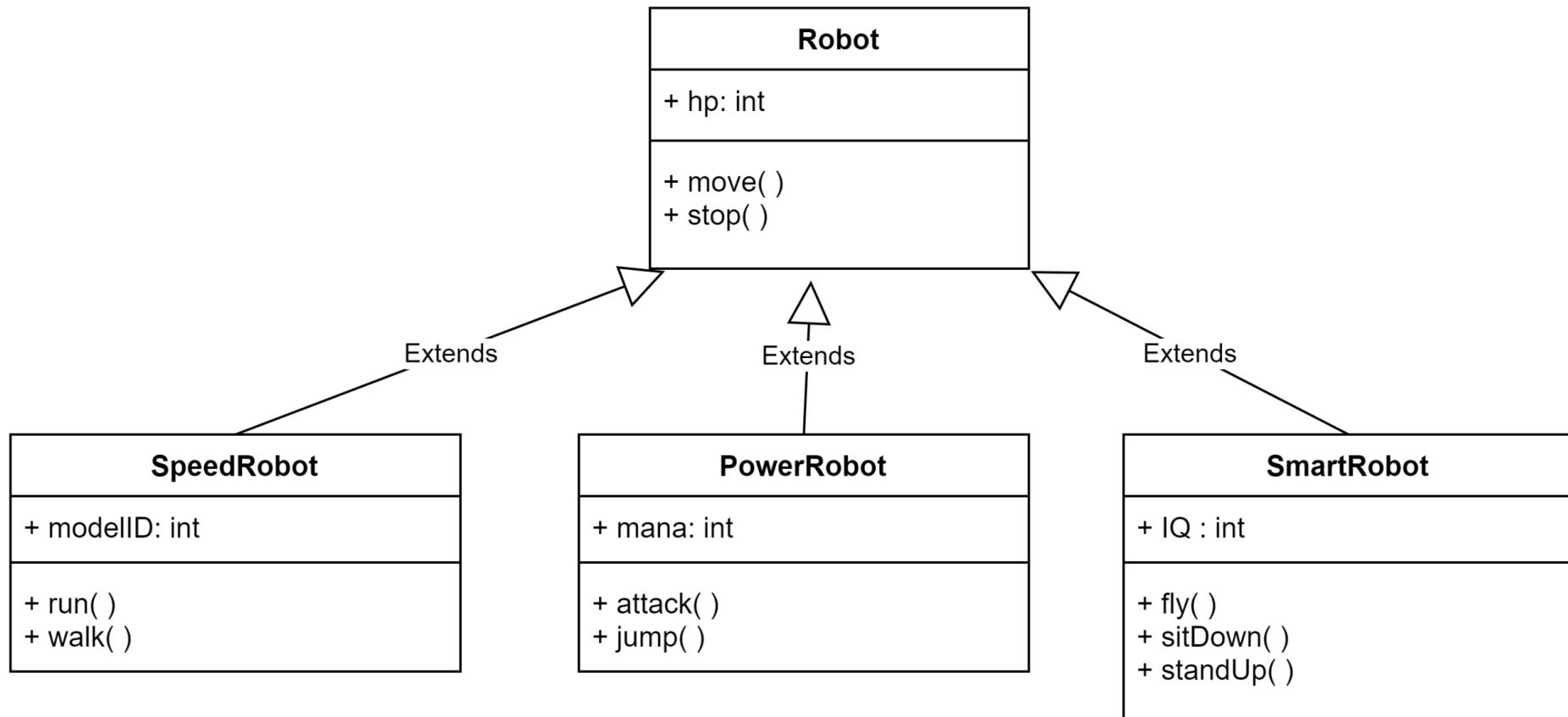
| SpeedRobot                                      |
|-------------------------------------------------|
| + hp: int<br>+ modelID: int                     |
| + move( )<br>+ stop( )<br>+ run( )<br>+ walk( ) |

| PowerRobot                                         |
|----------------------------------------------------|
| + hp: int<br>+ mana: int                           |
| + move( )<br>+ stop( )<br>+ attack( )<br>+ jump( ) |

| SmartRobot                                                         |
|--------------------------------------------------------------------|
| + hp: int<br>+ IQ : int                                            |
| + move( )<br>+ stop( )<br>+ fly( )<br>+ sitDown( )<br>+ standUp( ) |

# [도전] Generalization (일반화) 시키기

다음과 같은 코드로 수정하기



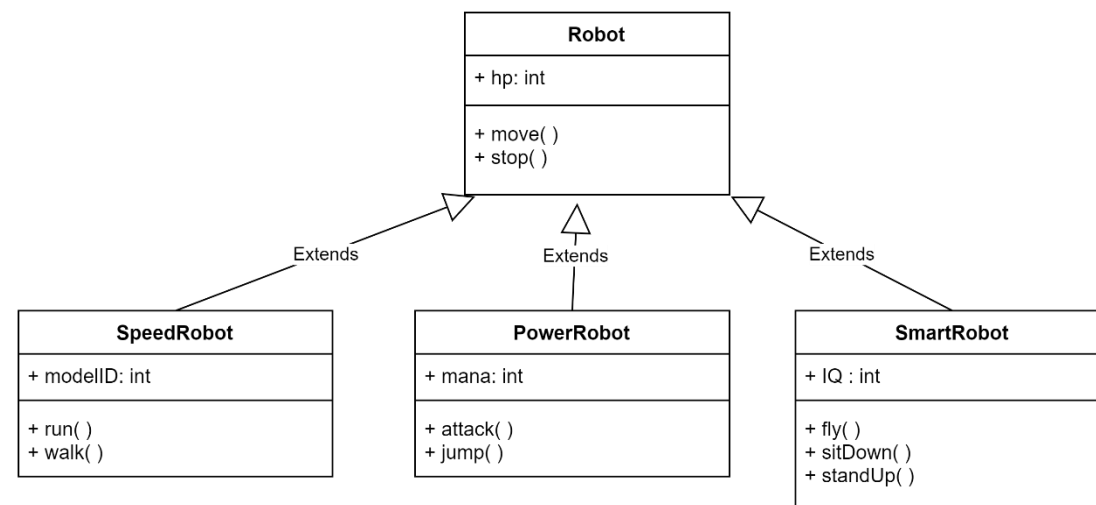


# 용어 정리

예시에 적합한 용어

**Super Class** (상위 클래스 - 상위 분류)

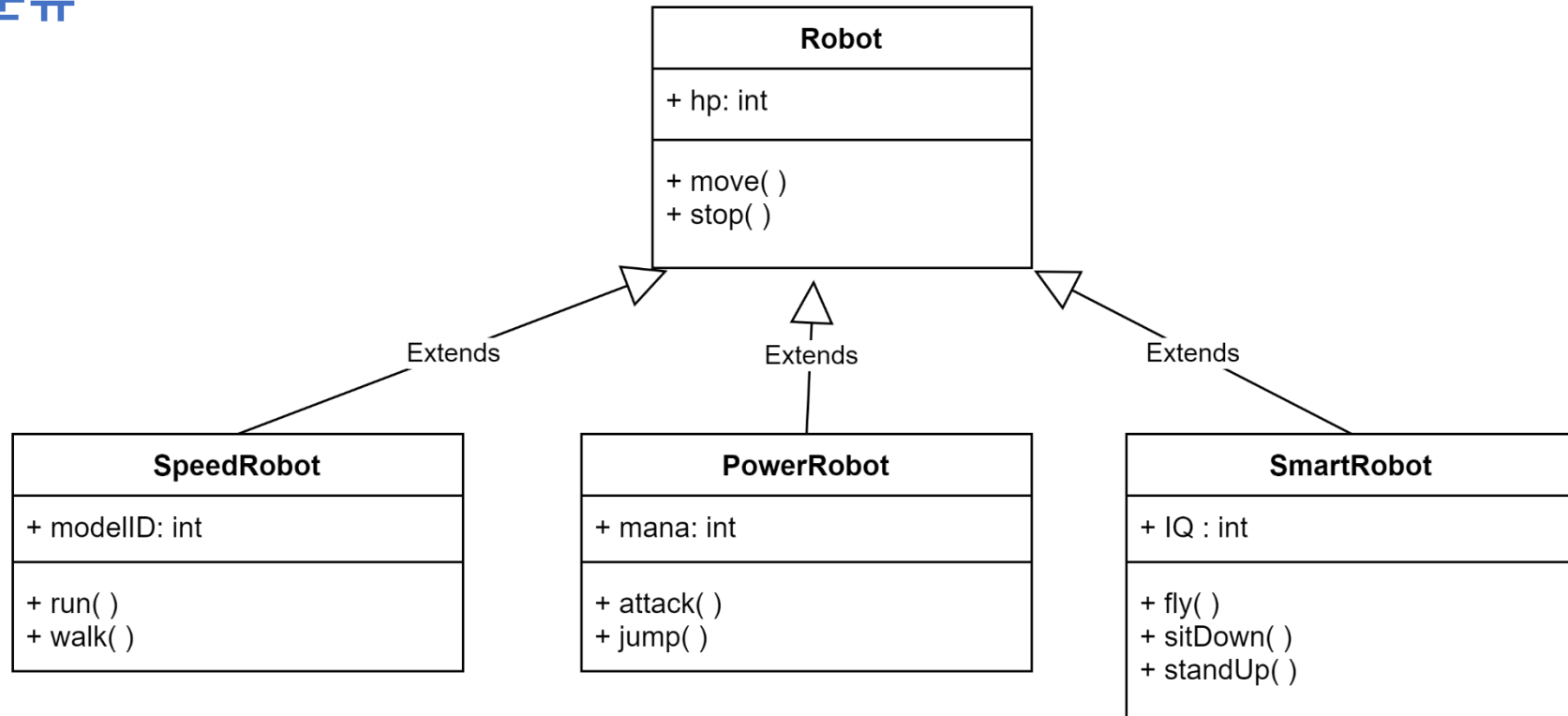
**Sub Class** (하위 클래스 - 하위 분류)



부모 클래스 / 자식 클래스라는 용어 보다 위 용어를 사용한다.

# 객체와 분류 관계

- “is a” 관계
- 국내 : “is a kind of” 관계
- 객체와 분류



# Overloading / Overriding

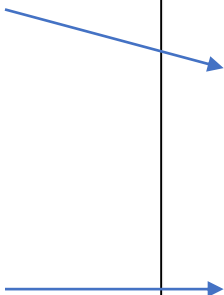
## Overriding

- Super Class 메서드 재정의

## Overloading

- 같은 이름의 메서드이지만,  
다른 Argument로 함수 구분

```
public class SpeedRobot extends Robot{  
  
    int modelID = 114;  
  
    void move() {  
        System.out.println("FAST RUN");  
    }  
  
    void move(int i) {  
        System.out.println("MOVE : " + i);  
    }  
  
    void run() { }  
  
    void walk() { }  
}
```



# [도전] Overriding

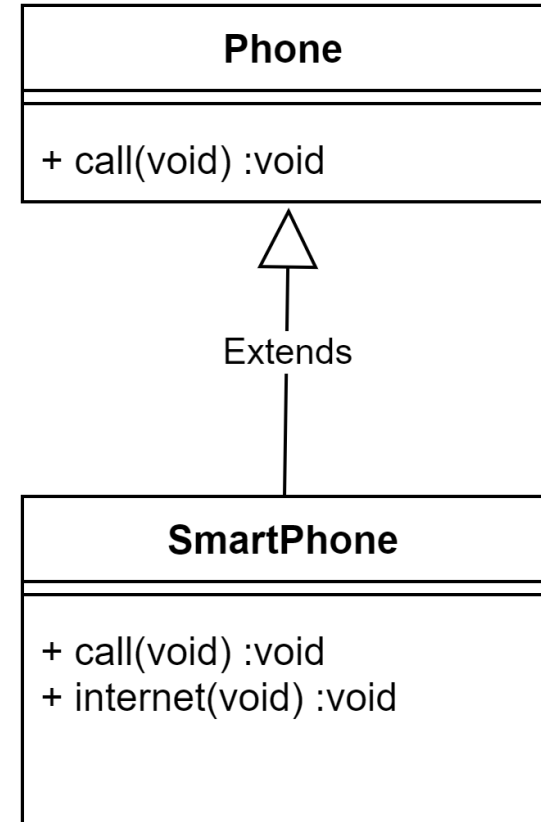
## 처음부터 구현해보기

### Phone Class의 call 메서드

- "Calling..." 출력

### SmartPhone Class의 call 메서드

- "SmartCalling..." 출력



# 어떤 값이 출력될까?

출력 결과를 예상해보자.

```
Phone a = new Phone();  
Phone b = new SmartPhone();  
SmartPhone c = new Phone();  
SmartPhone d = new SmartPhone();
```

- ① a.call();
- ② b.call();
- ③ c.call();
- ④ d.call();

## 다형성을 학습하는 이유

DI / Factory를 적용하기 위함!

DI, Factory를 통해 의존도, 의존복잡도를 낮출 수 있다.  
의존복잡도를 낮추면 유지보수성이 좋아진다.

## DI 적용하기

- 의존도(세기)를 낮출 수 있다.

## Factory 도입하기

- 의존복잡도를 낮출 수 있다.

## 구현방법 소개

- DI
- Factory

**OOP 에서 Interface의 개념잡기**

Interface 개요



# Interface 의미

## 접점

- 두개 시스템 사이의 경계면

# Interface 예시 1

TV 내부를 알지 못해도  
리모컨 Interface만 숙지하면 여러 TV를 제어할 수 있음



# Interface 예시 2

## USB Interface

- 컴퓨터 내부와 외부 장치를 연결할 수 있는 Interface
- Interface 규격에 맞는 장치를 개발하면, 어느 PC 에서도 연결 가능



# S/W Interface

## S/W Interface

- 내부에 접근하기 위한, 공통적인 형태
- 사용자는 Interface만 알고 있으면,  
쉽게 함수 사용 가능하다.

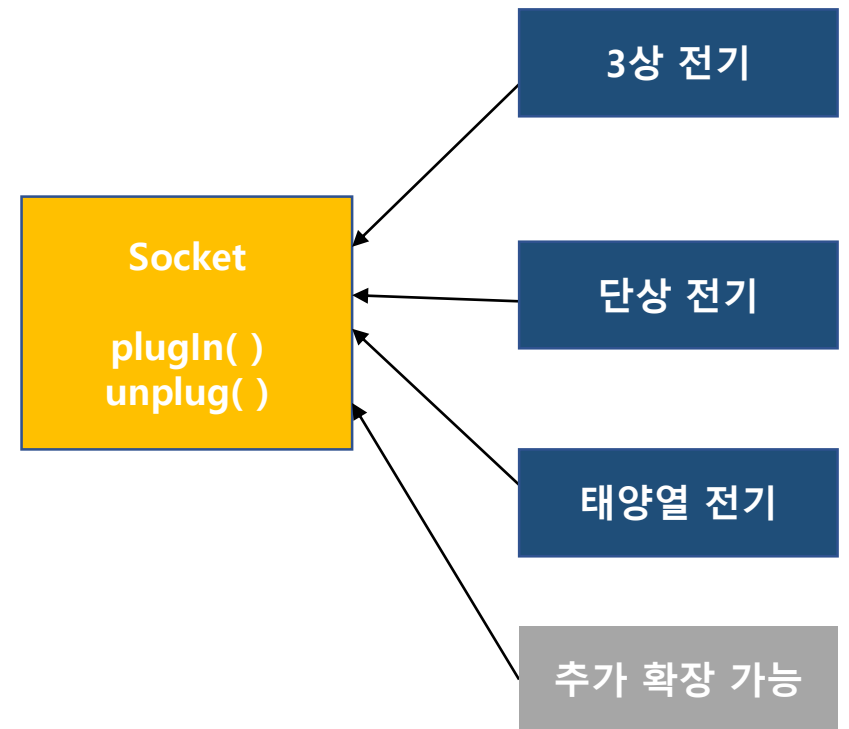


전기를 쉽게 사용하기 위한  
Interface 규격

# S/W Interface

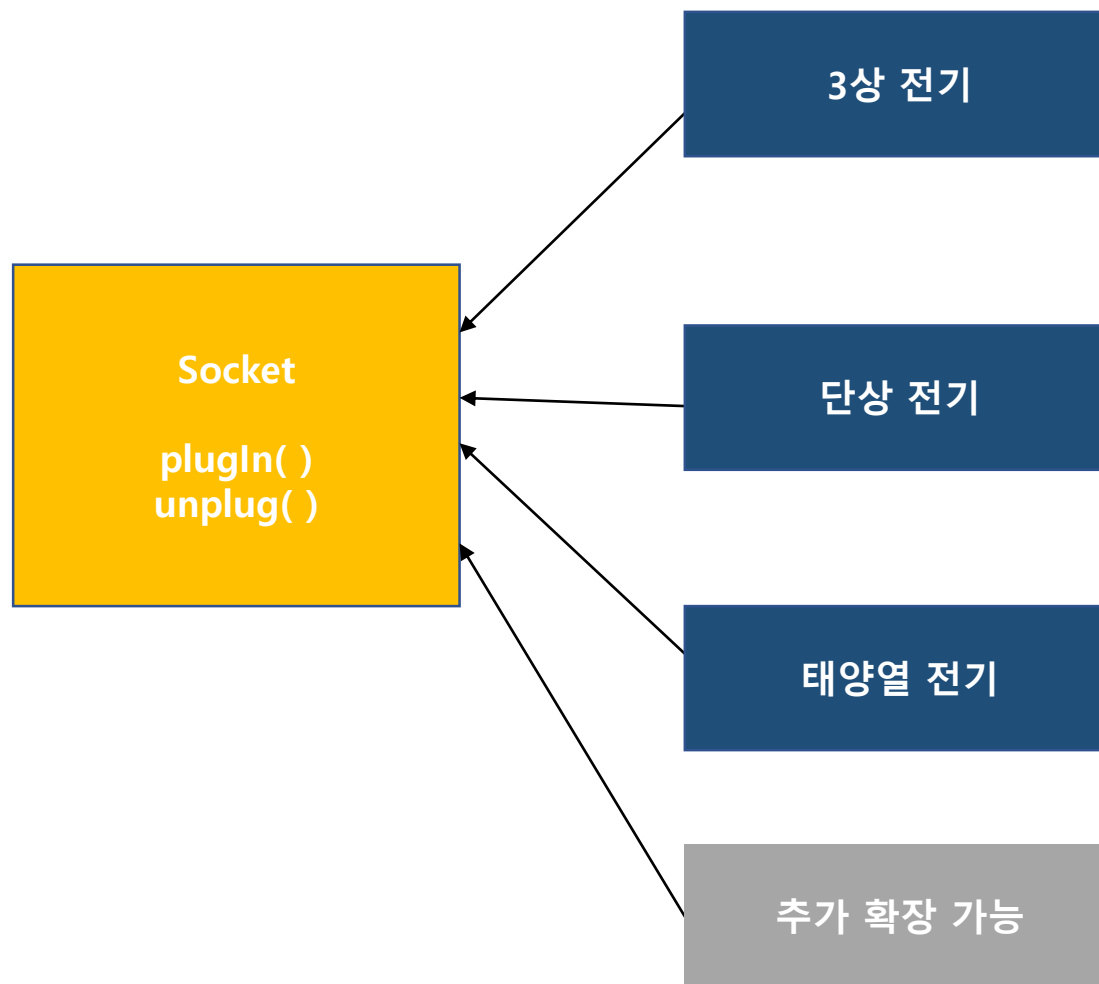
표준 규격을 만들어 놓고,  
표준 규격을 사용하는 Server 코드로 구현해 둬.

어떤 전기를 쓸지는 Client Code에서 선택



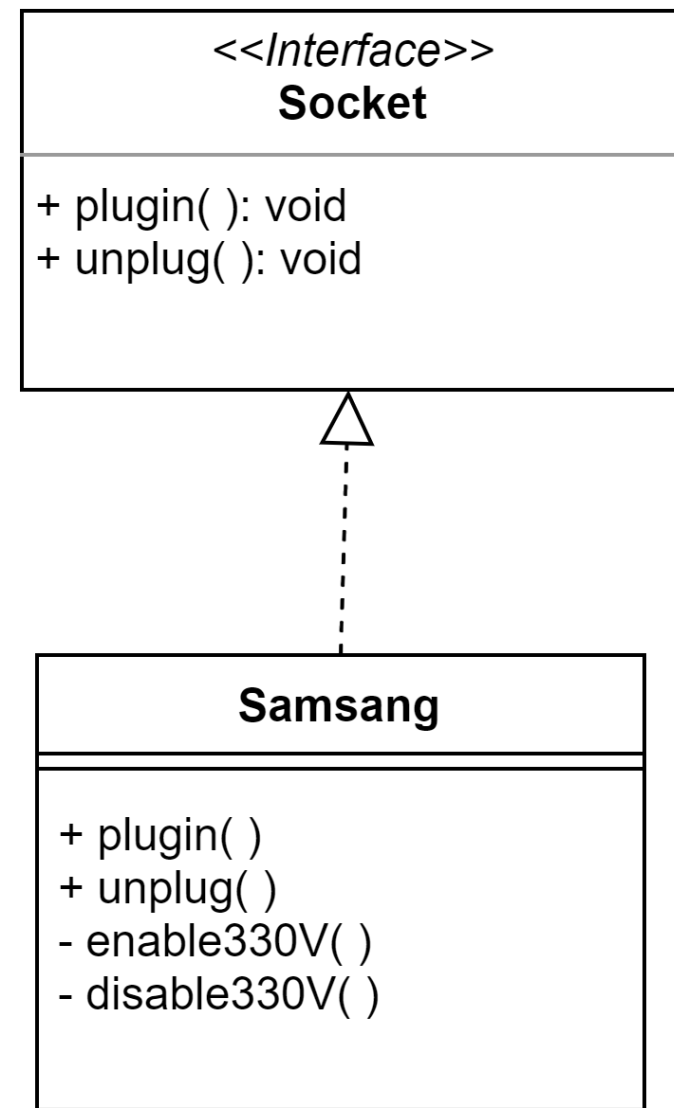
# Interface 를 쓰는 이유

1. 언젠가 추가될 유지보수를 위해  
확장 가능한 형태로 만듦
2. 객체를 쉽게 사용하고자  
표준화 시킴



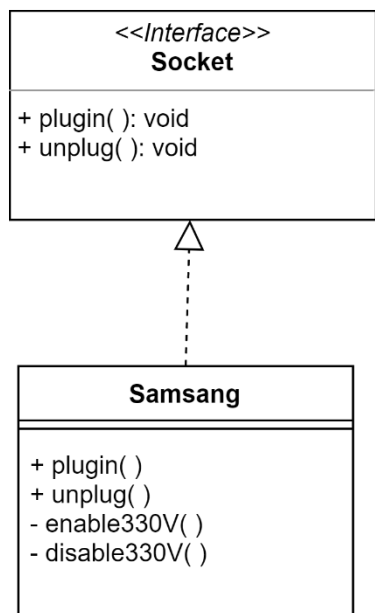
# Socket 구현하기

- ✓ Interface에 명시된 메서드는 반드시 구현해야 함
- ✓ Samsung은 Socket 이라는 Interface 규격을 따른다.
  - 이를, Realization 이라고 한다.



# Implements

## Interface를 실체화한 소스코드



```
public interface Socket {
    public void plugin();
    public void unplug();
}
```

```
public class Samsang implements Socket {

    @Override
    public void plugin() {
        enable330V();
    }

    @Override
    public void unplug() {
        disable330V();
    }

    private void enable330V() {
        System.out.println("3상 전기 연결");
    }

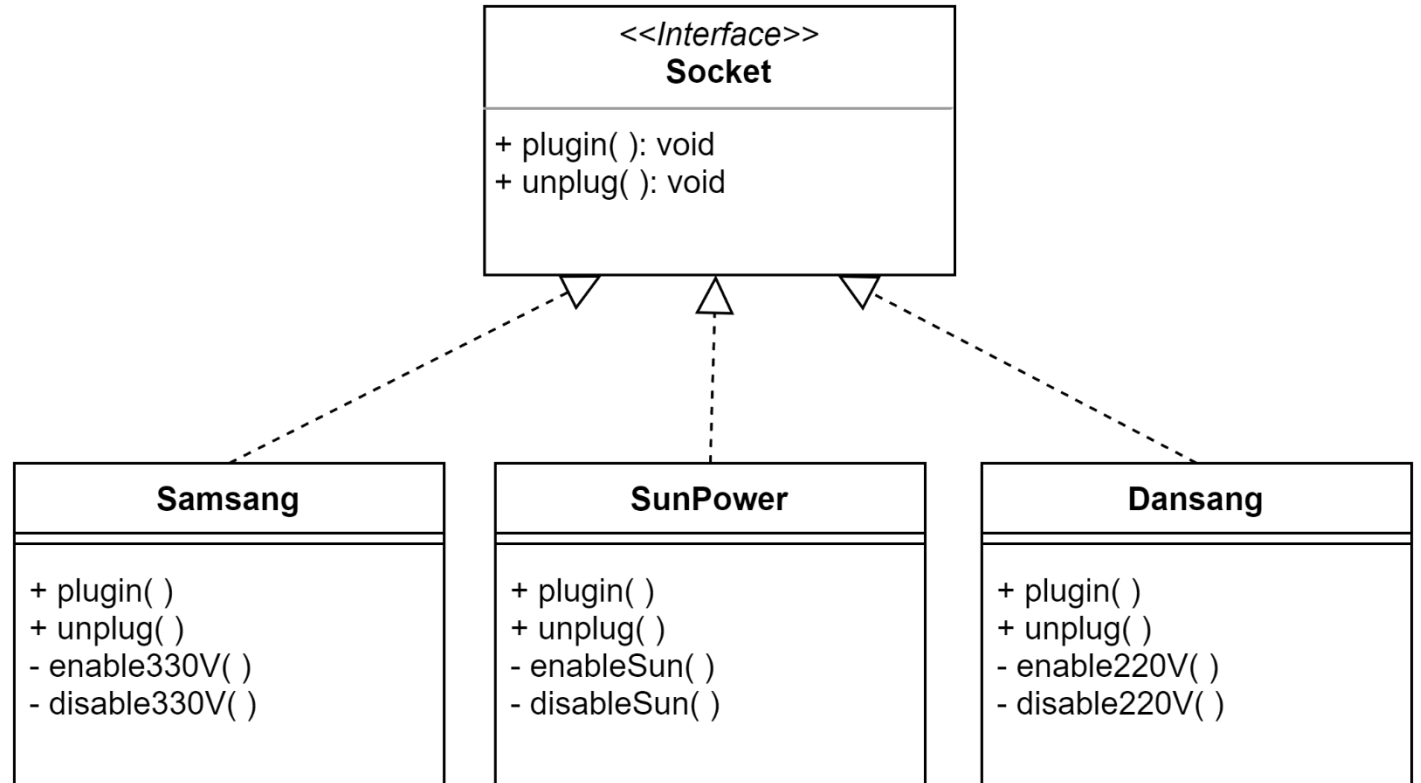
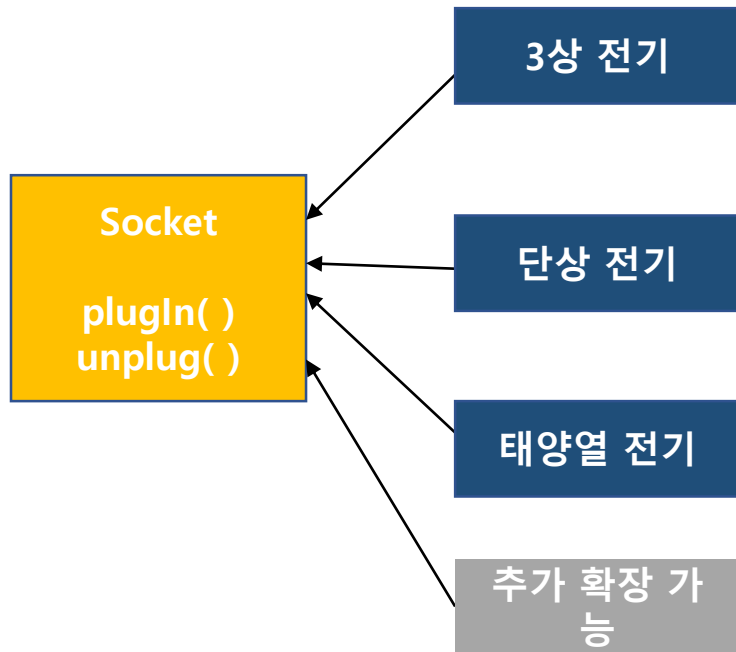
    private void disable330V() {
        System.out.println("3상 전기 해제");
    }

}
```



# [도전] Realization

SunPower / Dansang Class 모두 구현



**한 타입으로 다른 타입을 나타낼 수 있다.**

다형성 구현

# Client Code 이해

만든 객체들을 모두 생성하여, 메서드 호출

```
public static void main(String[] args) {  
  
    Samsang sam = new Samsang();  
    Dansang dan = new Dansang();  
    SunPower sp = new SunPower();  
  
    sam.plugin();  
    dan.plugin();  
    sp.plugin();  
}
```

# 다형성

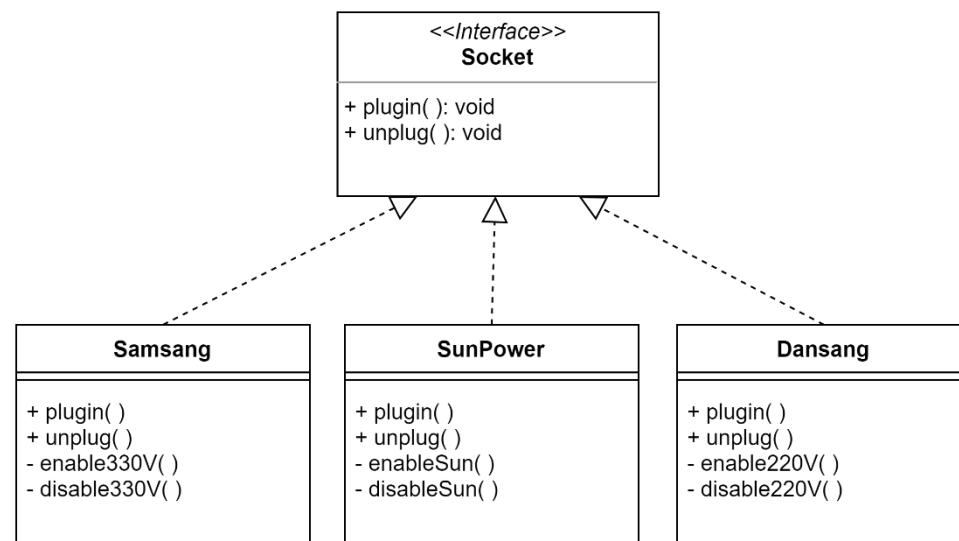
한 객체가 다양한 타입을 담을 수 있는 형태

```
public static void main(String[] args) {
```

```
    Socket sock1 = new Samsang();  
    Socket sock2 = new Dansang();  
    Socket sock3 = new SunPower();
```

```
    sock1.plugin();  
    sock2.plugin();  
    sock3.plugin();
```

```
}
```



# Client Server Code 예시

## plug 메서드

- Server Code
- 어떤 전기인지 상관없이,  
Interface에 맞게 동작 시킴

## main 메서드

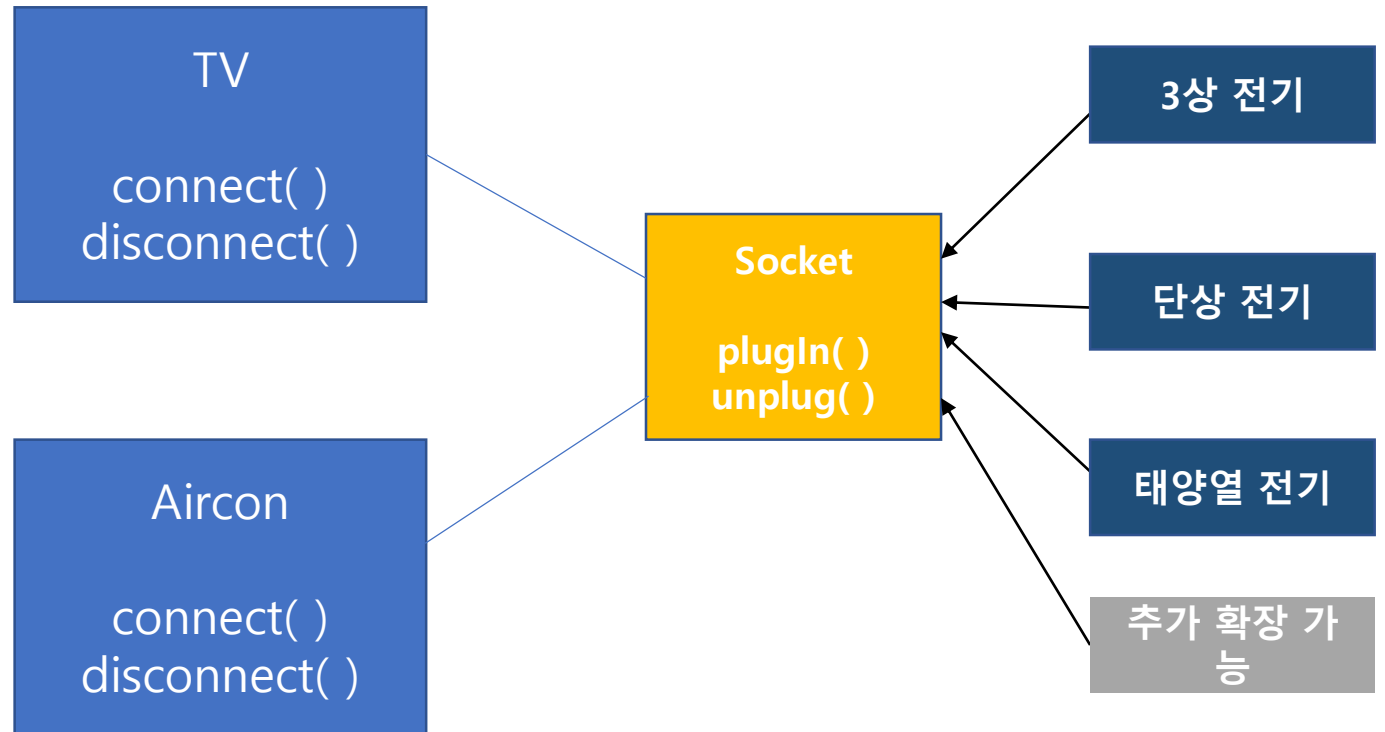
- client Code
- 어떤 전기를 쓸 지 결정함

```
public class Main {  
  
    //서버코드  
    static void dryMachine(Socket sock) {  
        System.out.println("전원 연결");  
        sock.plugin();  
        System.out.println("사용중...");  
        System.out.println("사용완료");  
        sock.unplug();  
    }  
  
    //클라이언트 코드  
    public static void main(String[] args) {  
        dryMachine(new Samsang());  
        dryMachine(new Dansang());  
        dryMachine(new SunPower());  
    }  
}
```

# [도전] 확장성 고려한 전자제품 구현하기

다양한 방법이 있음, 자유롭게 구현

- Server Code 작성
- Client Code 작성



확장성 고려한 Class 제작 Mission

# [실습] SortPrinter Machine

여러 수를 넣고, Sort를 수행해주는 머신

- 가능한 Sort
  - Bubble Sort
  - Selection Sort
- 어떤 Sort를 할지 선택을 한다.
- Run 버튼을 누르면, Sort가 진행된다.



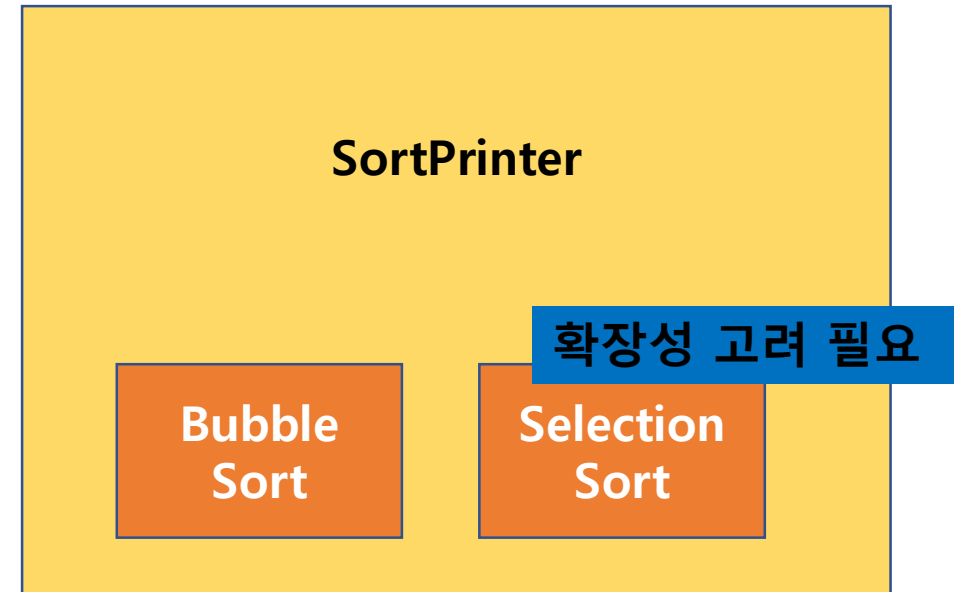


# [도전] SortPrinter Machine 구현하기

## 확장성을 고려한 머신 구현하기

### SortPrinter 기능

- insert 기능
  - SortPrinter 에 int 값들을 넣는다.
- selection 기능
  - 어떤 sort를 할지 선택한다.
- run 기능
  - 선택된 알고리즘이 수행된다.
- show 기능
  - 정렬된 결과가 출력된다.



감사합니다.